

GATED LATENT REASONING LOOP: A NOVEL PARADIGM FOR LATENT REASONING

Anonymous authors

Paper under double-blind review

ABSTRACT

We proposed a novel paradigm for implementing latent reasoning by combining DAPO-like reinforcement learning and special gating units. In this way, we transformed text-based explicit reasoning models to latent reasoning models, progressively and practically. Initial experiments demonstrate its feasibility, with significant stability and the least extra training overhead by parallelism and minimal changes, while holding full potential to scale up.

1 INTRODUCTION

The Chain of Continuous Thought (COCONUT) (Hao et al., 2024) explores the potential of the latent layer of reasoning as an alternative to the traditional explicit CoT (Wei et al., 2022) in both the reasoning procedure and the response procedure. Its previous work has shown that the latent layer of continuous reasoning can reduce unnecessary statements and focus on the core of the problem.

These models are issued from general-purpose language, trained in supervised steps of a reasoning procedure, and progressively increase their layers of latent reasoning. It is impractical due to the lack of customized reasoning data, and progressive training is significantly less convenient than explicit CoT. Hence, we provide a new approach to implementing this latent reasoning layer, fine-tuning from explicit reasoning models.

In 2014 (Chung et al., 2014), Chung et al. proposed the Gated Recurrent Unit. It is first leveraged to improve the RNNs, but we found that using it in the latent reasoning chain is also feasible to maintain its long-term contextual awareness.

In January, the DeepSeek team proposed a distillation-based method to implement a reasoning model through reinforcement learning (DeepSeek-AI, 2025), and applied the approach to Qwen-2.5B (Yang et al., 2024a) to a reasoning model. Given the limitations of a 2.5B parameter model in reasoning tasks, we aim to implement COCONUT on the small model and find its potential in reasoning compared to the original explicit reasoning model.

2 RELATED WORK

Explicit Reasoning Models and Large-scale Reinforcement Learning Advances in explicit reasoning models, including OpenAI o1 & o3 (OpenAI, 2025; 2024) and DeepSeek R1 (DeepSeek-AI, 2025), demonstrate their strong ability in mathematical reasoning. Based on Chain-of-Thought (Wei et al., 2022), just through prompts, models achieved significant improvements on arithmetic, commonsense, and symbolic reasoning tasks with *explicitly generate* intermediate reasoning text.

Several novel paradigms were also created from explicit reasoning models (Wang et al., 2023; Yao et al., 2023; Besta et al., 2024). In this way, explicit reasoning creates a new level of mathematical reasoning tasks.

Ouyang et al. (2022) showed how GPT-3 can be fine-tuned with reinforcement learning via Proximal Policy Optimization (PPO) (Schulman et al., 2017), a reinforcement learning algorithm that optimizes the agent’s policy by maximizing the expected reward while ensuring that the new policy does not deviate too much from the old policy. Shao et al. (2024) proposed Group Relative Policy Optimization (GRPO), which is a new reinforcement learning algorithm that overcame the limitations of PPO, including the need for a large number of samples and the difficulty of tuning

hyperparameters. Yu et al. (2025) demonstrated the effectiveness of Decoupled Clip and Dynamic Sampling Policy Optimization (DAPO) in large-scale reinforcement learning, which performed best in several benchmarks on Qwen2.5-32B Yang et al. (2024a) solely based on pure reinforcement learning and the base instruction model of Qwen.

In contrast, our work leverage a latent space instead of explicit reasoning text, which reduces verbosity and improves efficiency.

Chain of Continuous Thought Compared to explicit reasoning, latent reasoning paradigms leverage latent spaces, promoting efficiency and accuracy. However, they still face problems with scaling up and stability.

Previous works have already found that LLMs may involve hidden computation steps, which are not explicitly generated in text (Yang et al., 2024b; Chen et al., 2023). Driven by this spirit, Hao et al. (2024) proposed COCONUT, assigning models to “reason in a latent continuous space” through specifically designed training objectives, and demonstrated its potential compared to explicit Chain-of-Thought reasoning procedure, especially on tasks requiring backtracking.

Geiping et al. (2025) introduced a new recurrent transformer architecture that lets models perform multiple hidden reasoning iterations per token. A recurrent block can be looped arbitrarily at inference to refine the hidden state before producing the next token. Chen et al. (2025) argued that LLMs may involve a born ability to reason internally, suggesting that LLMs can internally learn better reasoning policies in the latent space when given appropriate training signals. Saunshi et al. (2025) demonstrated that the critical impact of model capabilities lies in its depth instead of parameter size. In this way, they proposed *Looped Transformer*, which generated latent thoughts and simulated Chain-of-Thought reasoning procedures internally.

In contrast, our work focuses on the practical implementation of latent reasoning, instead of cold-start training, which is done by Saunshi et al.. It excels in feasibility and stability, which places in the approach to train the model without specifically-designed trajectories, e.g., stepping guidance. It also combines the advances of explicit reasoning models and large-scale reinforcement learning objectives, while prior work mainly focused on how to produce the latent reasoning space.

3 METHODOLOGY

3.1 MODEL ARCHITECTURE

We first introduce the general structure of modern large language models (LLMs) to better demonstrate our approach and paradigm.

Modern LLMs involve the Embedding layer; Core Transformer, including Self-Attention and FFN; and the Output linear layer which project the latent state vector of `embed_dim` to probability distribution of `vocab_size` layers, as we denote them E , M_n , and O respectively, where n indicates the layer of hidden state (Vaswani et al., 2017; Radford et al., 2019).

3.2 INFERENCE

We used middle layers to enter the latent reasoning space to avoid breaking special usage in final layers (Elhage et al., 2021).

The whole inference procedure is described in (...)

We first perform prefilling by the question with the length q_0 into the model, and get the prefilled KV Cache (Pope et al., 2022) and the probability distribution of the answer’s first token t_{q_0+1} (starting to count from t_1).

After extracting the hidden state h_{n_0, q_0} (where n_0 denotes the layer number of the Transformer and q_0 denotes question length), we pass the state to a Variational Autoencoder (Kingma and Welling, 2019) (we will introduce it in 3.2.1), and produce a vector for storage with length l_s .

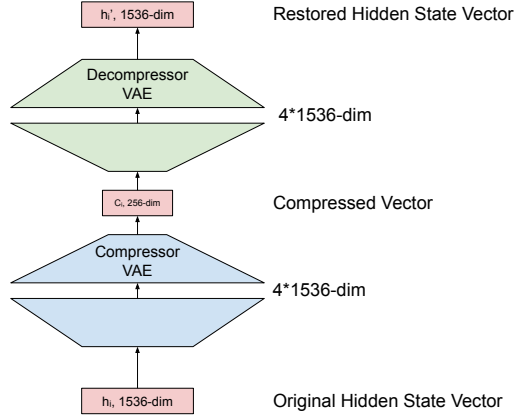


Figure 1: The VAE layer. We firstly compress the hidden state to a lower dimension (expand into 4 times of original size and then compress), extracting the compressed vector, and use the symmetric approach to expand it back to the original dimension.

3.2.1 VARIATIONAL AUTOENCODER LAYER

We use Variational Autoencoder (VAE) (Kingma and Welling, 2019) to compress and uncompress the hidden state token $h_{n,k}$, which is a l_h -dimensional vector, to a l_c -dimensional vector and then back to l_h .

The VAE layer is a bottleneck-like layer, which is used to compress the hidden state to a lower dimension and then expand it back to the original dimension. The application of VAE in our work can be described in Figure 1.

We pretrain the VAE on the online-generated activation of 1024 consequences, and the validation loss decreased steadily, proving that the pretraining process of the VAE does not require much data. We will provide supplemental data demonstrating its effectiveness in the appendix.

Long-term Cache During sampling, we will store the latent space activation outputted from later n_0 for further paralleled training. Yet directly storing the raw latent vector of embed_dim could be space consuming. (See ¹ for quantitative analysis of the space consumption).

To enable long-term storage of reasoning features, we introduce a VAE-style bottleneck layer, compressing 1536-dimensional features to 256 dimensions and then expanding back to 1536 before gating.

We store it in each operation, and concatenate these vectors into a matrix subsequently as a hidden state.

3.2.2 GATING MECHANISM

Our model has two gates: the general gate, the injection scalar, and the Dynamic Current Step Mixing Gate.

1. **General Gate.** The general gate is derived from the sequence; its value is dynamic. We denote it as g_t , and its derivation procedure can be described in Equation (not yet). We initialize it to all zeros, i.e., initially the model is equivalent to the original one, and it evolves as g moves away from 0.
2. **Dynamic Current-Step-Mixing Gate (DMG).**

¹Consider a sample batch size of 1024, sequence length 1024, and embedding dimension 1536. Each element occupies 2 bytes (BF16), resulting in $1024 \times 1024 \times 1536 \times 2 = 2^{31.5}$ bytes, or approximately 2.8 GB of VRAM.

The processed vector $\mathbf{h}'_{n,k}$ after the VAE layer is passed to several gates, as we denote in Equation 1.

$$\mathbf{h}'_{n,k} = \mathbf{h}'_{n,k-1} \odot \mathbf{g} \odot \text{DMG}(\mathbf{e}_k) \times s_{\text{inj}} + (1 - \mathbf{g}) \odot \mathbf{e}_k \quad (1)$$

where $\mathbf{e}_k = \text{Embedding}(\mathbf{t}_k)$; $\mathbf{t}_k$ denotes the token of sequence with position k , and s_{inj} denotes injection scalar, which is a constant scalar to bridge the difference in order of magnitude among gating value and the result of Dynamic Current-Step-Mixing Gate, which will be described in Section 3.2.2.

3.3 TRAINING

We use the DAPO objective as our reinforcement learning framework, but the whole procedure is slightly different from the original DAPO. The whole training procedure is described in Algorithm 1.

Algorithm 1: Training Procedure taking one batch as an example.

Data: Given a sequence S of seq_length, where $\mathbf{t}_k \in S$; $\mathbf{t}_k$ denotes token index.

Result: Model

```

1  $\mathbf{e} \leftarrow \text{Embedding}(\mathbf{s}[: -1]);$ 
2  $\mathbf{h} \leftarrow \mathbf{E}(\mathbf{e});$ 
3  $p \leftarrow \text{Softmax}(\mathbf{O}(\mathbf{h}));$ 
4  $\text{loss} \leftarrow \text{CrossEntropyLoss}(p, \text{one\_hot}([: -1]));$ 
5 Optimize by minimizing  $\text{loss}$ ;
```

3.3.1 REINFORCEMENT LEARNING

The whole procedure of reinforcement learning can be described in Figure.

We adapt the DAPO objective, which is formulated as:

$$\begin{aligned} \mathcal{J}_{\text{DAPO}}(\theta) = & \mathbb{E}_{(q,a) \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|q)} \\ & \frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1}^G \sum_{t=1}^{|o_i|} r_{i,t}(\theta) \hat{A}_{i,t} \end{aligned} \quad (2)$$

subject to $0 < |\{o_i \mid \text{is_equivalent}(a, o_i)\}| < G$,

where

$$r_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})}, \quad \hat{A}_{i,t} = \frac{R_i - \text{mean}(\{R_i\}_{i=1}^G)}{\text{std}(\{R_i\}_{i=1}^G)}. \quad (3)$$

Firstly we apply rigorous datasets to the inference procedure, and verify the answer (Kydliček).

We parallelized several inference procedures simultaneously, preferably 16 for a same question, and verify the answer.

Award Mechanism We use $\text{reward}_{\text{acc}}$ and $\text{penalty}_{\text{len}}$ to award or penalize in accuracy, according to DAPO (Yu et al., 2025). The exact value of the reward and penalty will be described in appendices.

Positive-Negative Clipping To eliminate the problem of too much negative samples (which each holds less use to the model’s improvement) under a budget-tight scenario, we only choose all the positive samples, and randomly choose the rest negative samples according to the ratio 1 : 1.

3.3.2 TIME CONSISTENCY REGULARIZATION

We adapt a mechanism called “Gating Value Bonus,” which is used to guarantee the gate to converge to a fixed position to let the model make use of the information from latent space instead of setting stuck in the local optima of not using the space sufficiently.

The gating value bonus is a regularization term that encourages the gating values to be close to the middle, as the formulation is shown in Equation 4.

$$\mathcal{J}_{\text{gate}}(\mathbf{g}) = \frac{1}{\text{output_dim}} C_0 \sum_{i=0}^{\text{output_dim}} \exp \left[\lambda \left(\frac{1}{2} - g_i \right)^2 \right] \quad (4)$$

where $g_i = \mathbf{g} \cdot \mathbf{e}^{(i)}$, i.e., the i -th element of the gate vector $\mathbf{g}$, and C_0 is a constant to control the scale of the bonus; λ is a constant to control the rate of convergence.

3.3.3 PARALLELIZED TRAINING

After sampling, we concatenate the question, answer, and hidden state, and apply it to training procedure.

4 EXPERIMENTS

4.1 SETUP

Base Model We use Qwen2.5-1.5B (Yang et al., 2024a) as our base model, which is a 1.5B parameter model with 24 layers. DeepSeek distilled the model into a explicit reasoning model (DeepSeek-AI, 2025), which is our comparison baseline.

Baselines We compare our model with the following baselines:

- **Qwen2.5-1.5B**: The original Qwen2.5-1.5B model.
- **DeepSeek-R1-distill-Qwen2.5-1.5B**: The explicit reasoning model distilled from Qwen2.5-1.5B by DeepSeek (DeepSeek-AI, 2025).
- The original model with DAPO (Yu et al., 2025) objective solely.
- Our model with DAPO objective, gating mechanism, and VAE layer.
- The original model with VAPO (Yue et al., 2025) objective solely.

5 DISCUSSION

6 CONCLUSION

REFERENCES

- M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk, and T. Hoefler. Graph of thoughts: Solving elaborate problems with large language models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16):17682–17690, Mar. 2024. ISSN 2159-5399. doi: 10.1609/aaai.v38i16.29720. URL <http://dx.doi.org/10.1609/aaai.v38i16.29720>.
- H. Chen, Y. Feng, Z. Liu, W. Yao, A. Prabhakar, S. Heinecke, R. Ho, P. L. Mui, S. Savarese, C. Xiong, and H. Wang. Language models are hidden reasoners: Unlocking latent reasoning capabilities via self-rewarding, 2025. URL <https://openreview.net/forum?id=4Po8d9GAfQ>.
- W. Chen, M. Yin, M. Ku, P. Lu, Y. Wan, X. Ma, J. Xu, X. Wang, and T. Xia. Theoremqa: A theorem-driven question answering dataset, 2023. URL <https://arxiv.org/abs/2305.12524>.
- J. Chung, Ç. Gülçehre, K. Cho, and Y. Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *CoRR*, abs/1412.3555, 2014. URL <http://arxiv.org/abs/1412.3555>.
- DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.

- N. Elhage, N. Nanda, C. Olsson, T. Henighan, N. Joseph, B. Mann, A. Askell, Y. Bai, A. Chen, T. Conerly, N. DasSarma, D. Drain, D. Ganguli, Z. Hatfield-Dodds, D. Hernandez, A. Jones, J. Kernion, L. Lovitt, K. Ndousse, D. Amodei, T. Brown, J. Clark, J. Kaplan, S. McCandlish, and C. Olah. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021. URL <https://transformer-circuits.pub/2021/framework/index.html>.
- J. Geiping, S. McLeish, N. Jain, J. Kirchenbauer, S. Singh, B. R. Bartoldson, B. Kailkhura, A. Bhatele, and T. Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach, 2025. URL <https://arxiv.org/abs/2502.05171>.
- S. Hao, S. Sukhbaatar, D. Su, X. Li, Z. Hu, J. Weston, and Y. Tian. Training large language models to reason in a continuous latent space, 2024. URL <https://arxiv.org/abs/2412.06769>.
- D. P. Kingma and M. Welling. An introduction to variational autoencoders. *CoRR*, abs/1906.02691, 2019. URL <http://arxiv.org/abs/1906.02691>.
- H. Kydlíček. Math-Verify: Math Verification Library. URL <https://github.com/huggingface/math-verify>.
- OpenAI. Introducing openai o1, September 2024. URL <https://openai.com/o1/>. Accessed: 2025-04-27.
- OpenAI. Introducing openai o3 and o4-mini, April 2025. URL <https://openai.com/index/introducing-o3-and-o4-mini/>. Accessed: 2025-04-27.
- L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe. Training language models to follow instructions with human feedback, 2022. URL <https://arxiv.org/abs/2203.02155>.
- R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, A. Levskaya, J. Heek, K. Xiao, S. Agrawal, and J. Dean. Efficiently scaling transformer inference, 2022. URL <https://arxiv.org/abs/2211.05102>.
- A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019. URL https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.
- N. Saunshi, N. Dikkala, Z. Li, S. Kumar, and S. J. Reddi. Reasoning with latent thoughts: On the power of looped transformers, 2025. URL <https://arxiv.org/abs/2502.17416>.
- J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347, 2017. URL <http://arxiv.org/abs/1707.06347>.
- Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL <https://arxiv.org/abs/2402.03300>.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. *CoRR*, abs/1706.03762, 2017. URL <http://arxiv.org/abs/1706.03762>.
- X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou. Self-consistency improves chain of thought reasoning in language models, 2023. URL <https://arxiv.org/abs/2203.11171>.
- J. Wei, X. Wang, D. Schuurmans, M. Bosma, E. H. Chi, Q. Le, and D. Zhou. Chain of thought prompting elicits reasoning in large language models. *CoRR*, abs/2201.11903, 2022. URL <https://arxiv.org/abs/2201.11903>.

- A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Lin, K. Dang, K. Lu, K. Bao, K. Yang, L. Yu, M. Li, M. Xue, P. Zhang, Q. Zhu, R. Men, R. Lin, T. Li, T. Xia, X. Ren, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Wan, Y. Liu, Z. Cui, Z. Zhang, and Z. Qiu. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024a.
- S. Yang, E. Gribovskaya, N. Kassner, M. Geva, and S. Riedel. Do large language models latently perform multi-hop reasoning?, 2024b. URL <https://arxiv.org/abs/2402.16837>.
- S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. Tree of thoughts: Deliberate problem solving with large language models, 2023. URL <https://arxiv.org/abs/2305.10601>.
- Q. Yu, Z. Zhang, R. Zhu, Y. Yuan, X. Zuo, Y. Yue, T. Fan, G. Liu, L. Liu, X. Liu, H. Lin, Z. Lin, B. Ma, G. Sheng, Y. Tong, C. Zhang, M. Zhang, W. Zhang, H. Zhu, J. Zhu, J. Chen, J. Chen, C. Wang, H. Yu, W. Dai, Y. Song, X. Wei, H. Zhou, J. Liu, W.-Y. Ma, Y.-Q. Zhang, L. Yan, M. Qiao, Y. Wu, and M. Wang. Dapo: An open-source llm reinforcement learning system at scale, 2025. URL <https://arxiv.org/abs/2503.14476>.
- Y. Yue, Y. Yuan, Q. Yu, X. Zuo, R. Zhu, W. Xu, J. Chen, C. Wang, T. Fan, Z. Du, X. Wei, X. Yu, G. Liu, J. Liu, L. Liu, H. Lin, Z. Lin, B. Ma, C. Zhang, M. Zhang, W. Zhang, H. Zhu, R. Zhang, X. Liu, M. Wang, Y. Wu, and L. Yan. Vapo: Efficient and reliable reinforcement learning for advanced reasoning tasks, 2025. URL <https://arxiv.org/abs/2504.05118>.

A APPENDIX